



# Arrays & Linked Lists



# Arrays

# IOIO

# Introduction to Arrays

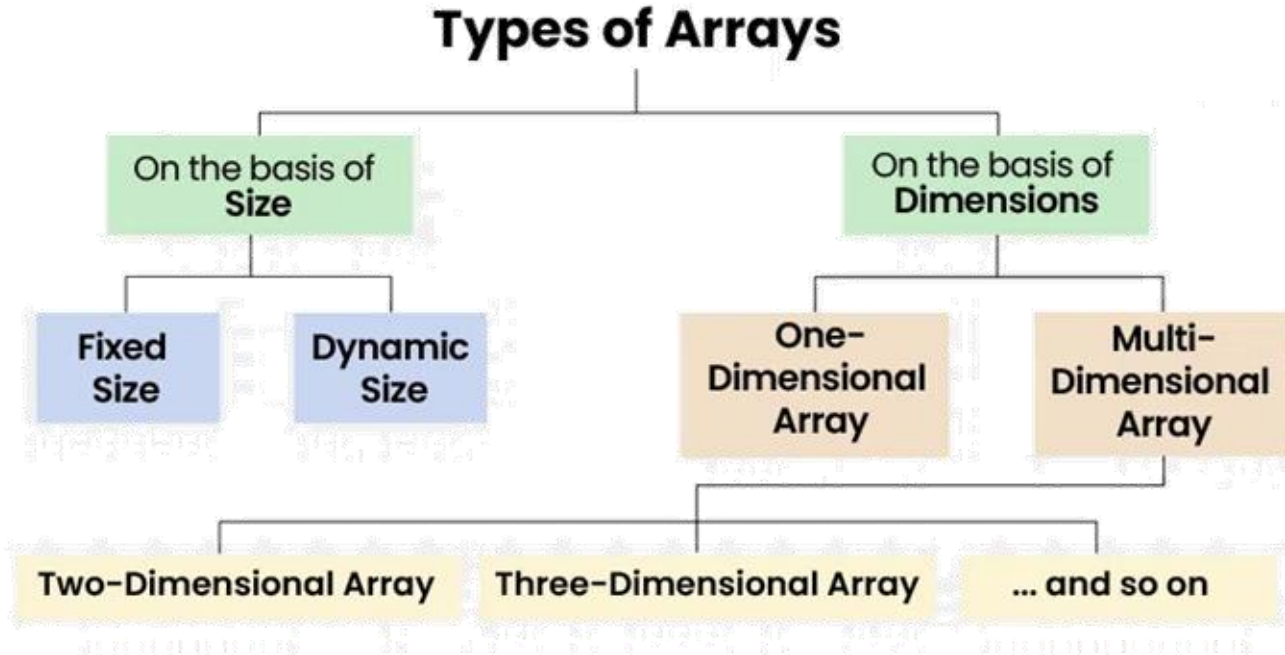
- **Definition**

- An array is a collection of elements, each identified by an index or a key
- Fundamental data structure that stores elements of the same type
- Uses contiguous memory locations

- **Characteristics**

- Fixed size (in most languages)
- Elements are stored in contiguous memory
- Constant-time access to elements
- Homogeneous data type

# Types of Arrays



# Memory Representation of Arrays

- **Contiguous Memory Allocation**

- Arrays occupy a continuous block of memory, with each element taking up the same amount of space.

- **Index-Based Access**

- Elements are accessed using their index, which represents their position relative to the start of the array.

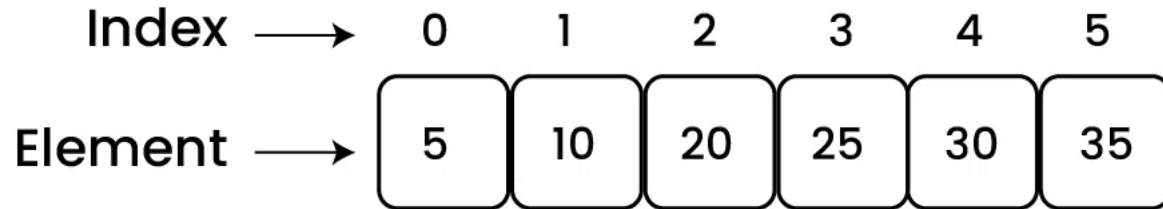
- **Memory Address Calculation**

- The memory address of an element can be calculated using the formula:

$$\text{memory\_address} = \text{base\_address} + (\text{index} * \text{size\_of\_element})$$

# One-Dimensional Array

## One-Dimensional Array (1-D Array)



# Array Operations

- **Access**

- Retrieve an element at a given index.
- Time complexity:  $O(1)$

- **Update**

- Modify an element at a specific index.
- Time complexity:  $O(1)$

- **Insert**

- Add a new element which may require shifting elements.
- Time complexity:  $O(n)$

- **Delete**

- Remove an element which may require shifting elements.
- Time complexity:  $O(n)$

# Array Common Problems

- **Reverse an array**

- Given an array of integers, reverse the order of its elements. You can use a loop to iterate through the array and swap the elements at the beginning and end, then move inwards towards the center.

- **Find the maximum element**

- Find the largest element in an array. Iterate through the array and keep track of the maximum element encountered so far.

- **Check if an array is sorted**

- Determine if the elements in an array are in ascending or descending order. You can iterate through the array and compare adjacent elements. If any pair is not in the correct order, the array is not sorted.



# Reverse an Array

## Java

```
public static void reverseArray(int[] arr) {  
    int n = arr.length;  
    for (int i = 0; i < n / 2; i++) {  
        int temp = arr[i];  
        arr[i] = arr[n - i - 1];  
        arr[n - i - 1] = temp;  
    }  
}
```

## Python

```
def reverse_array(arr):  
    n = len(arr)  
    for i in range(n // 2):  
        arr[i], arr[n - i - 1] = arr[n - i - 1], arr[i]
```

# Find the Maximum Element

## Java

```
public static int findMax(int[] arr) {  
    int max = arr[0];  
    for (int i = 1; i < arr.length; i++) {  
        if (arr[i] > max) {  
            max = arr[i];  
        }  
    }  
    return max;  
}
```

## Python

```
def find_max(arr):  
    max_val = arr[0]  
    for val in arr[1:]:  
        if val > max_val:  
            max_val = val  
    return max_val
```

# Check if an Array is Sorted

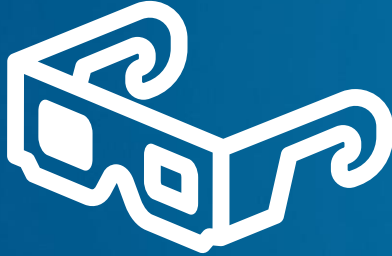
## Java

```
public static boolean isSorted(int[] arr) {
    if (arr.length <= 1) {
        return true;
    }
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] < arr[i - 1]) {
            return false;
        }
    }
    return true;
}
```

## Python

```
def is_sorted(arr):
    if len(arr) <= 1:
        return True
    for i in range(1, len(arr)):
        if arr[i] < arr[i - 1]:
            return False
    return True
```

# Multi-Dimensional Arrays



# Introduction to Multi-dimensional Arrays

- **Definition**

- Multi-dimensional arrays are arrays of arrays
- Tables with rows and columns

- **Common Uses**

- Representing matrices
- Grid-based games
- Image processing

# Multi-Dimensional Arrays

## Two-Dimensional Array (2-D Array or Matrix)

Columns →

Rows ↓

|   | 0        | 1        | 2        |
|---|----------|----------|----------|
| 0 | $a_{00}$ | $a_{01}$ | $a_{02}$ |
| 1 | $a_{10}$ | $a_{11}$ | $a_{12}$ |
| 2 | $a_{20}$ | $a_{21}$ | $a_{22}$ |

## Three-Dimensional Array (3-D Array)

Columns

Column 1

Column 2

Column 3

Rows

Row 1

Row 2

Row 3

Matrix 1

Matrix 2

Matrix 3

Matrices

|  |           |           |           |           |           |
|--|-----------|-----------|-----------|-----------|-----------|
|  | $a_{000}$ | $a_{001}$ | $a_{002}$ |           |           |
|  | $a_{010}$ | $a_{100}$ | $a_{101}$ | $a_{102}$ |           |
|  | $a_{020}$ | $a_{110}$ | $a_{200}$ | $a_{201}$ | $a_{202}$ |
|  |           | $a_{120}$ | $a_{210}$ | $a_{211}$ | $a_{212}$ |
|  |           |           | $a_{220}$ | $a_{221}$ | $a_{222}$ |

# 2D Array Common Problems

- **2D Array Traversal**

- Given a 2D array, loop through every element using nested loops.

- **Find the Smallest Element in 2D Array**

- Given a 2D array, find the smallest element.
- Iterate through the array, comparing each element to the current minimum.
- Update the minimum if a smaller value is found.

# 2D Array Traversal

## Java

```
public void traverse(int[][] matrix) {  
    for (int i = 0; i < matrix.length; i++) {  
        for (int j = 0; j < matrix[i].length; j++) {  
            System.out.print(matrix[i][j] + " ");  
        }  
        System.out.println();  
    }  
}
```

## Python

```
def traverse(matrix):  
    for row in matrix:  
        for value in row:  
            print(value, end=' ')  
        print()
```



# Find the Smallest Element in 2D Array

## Java

```
public static int findSmallestElement(int[][] arr) {  
    int smallest = arr[0][0];  
    for (int[] row : arr) {  
        for (int element : row) {  
            if (element < smallest) {  
                smallest = element;  
            }  
        }  
    }  
    return smallest;  
}
```

## Python

```
def find_smallest_element(matrix):  
    smallest = matrix[0][0]  
    for row in matrix:  
        for element in row:  
            if element < smallest:  
                smallest = element  
    return smallest
```

# Linked Lists



# Introduction to Linked Lists

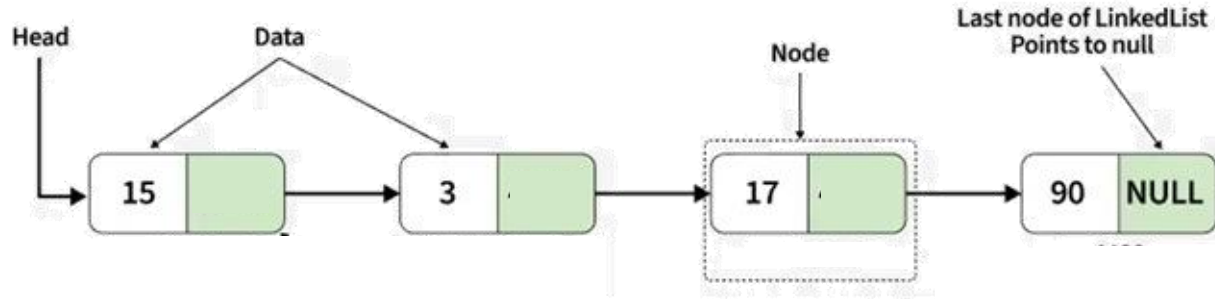
- **Definition**

- Linear data structure where elements are stored in nodes.
- Each node contains data field and reference (link) to next node in sequence.

- **Characteristics**

- Dynamic size
- Efficient insertion and deletion
- Non-contiguous memory allocation
- Sequential access

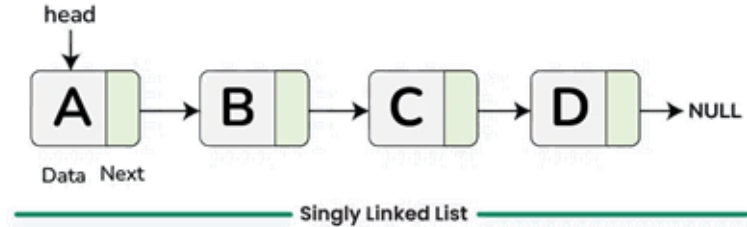
# Linked List



# Types of Linked Lists

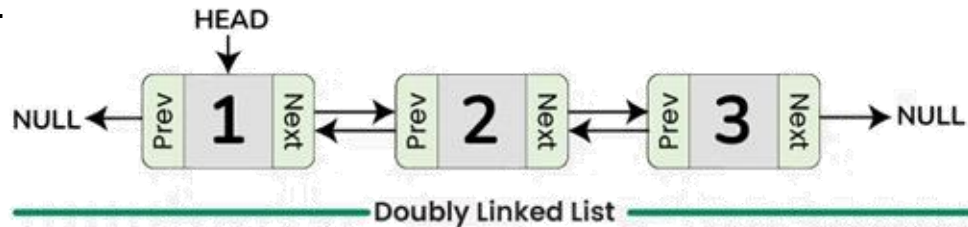
- **Singly Linked List**

- Each node has data field and 1 link to the next node.
- Traversal is possible in one direction only.



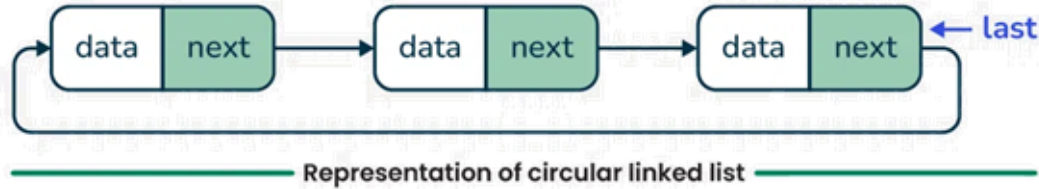
- **Doubly Linked List**

- Each node has data field and 2 links - 1 to next node and 1 to previous node.
- Allows bi-directional traversal.

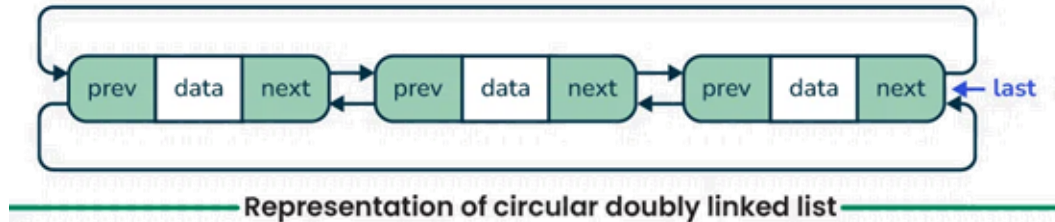


# Circular Linked Lists

- Circular Linked List has last item pointing back to beginning



- Circular Double Linked List has last item pointing back to beginning and first item's previous points to the end



# Singly Linked List & Node Structure

## Java

```
// Singly Linked Node
class Node {
    int data;
    Node next;
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// Singly Linked List
class LinkedList {
    Node head;
    public LinkedList() {
        this.head = null;
    }
}
```

## Python

```
# Singly Linked Node
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Singly Linked List
class LinkedList:
    def __init__(self):
        self.head = None
```

# Linked List Operations

- **Search**

- Find a node with a specific value.
- Time complexity:  $O(n)$

- **Insert**

- Add a new node at the beginning, middle, or end
- Time complexity:  $O(1)$  or  $O(n)$

- **Delete**

- Remove a node from the list.
- Time complexity:  $O(1)$  or  $O(n)$

- **Traverse**

- Visit each node in the list.
- Time complexity:  $O(n)$



# Insertion of Element into Position in Linked List

- Create new node and set its data
- If Position = 1 (*adding at beginning*):
  - Set new node's next pointer to current head
  - Update head to point to the new node
- If Position = N (*adding at end*):
  - Update last node's next pointer to the new node
- If Position > 1 and < N (*adding in middle*):
  - Traverse list to find where to add node
  - Set previous' next pointer to new node
  - Set new node's next pointer to next node

# Insertion at the Beginning of Linked List

## Java

```
public void insertAtBeginning(int new_data) {  
    Node new_node = new Node(new_data);  
    new_node.next = head;  
    head = new_node;  
}
```

## Python

```
def insert_at_beginning(self, new_data):  
    new_node = Node(new_data)  
    new_node.next = self.head  
    self.head = new_node
```

# Insertion at the End of Linked List

## Java

```
public void insertAtEnd(int new_data) {
    Node new_node = new Node(new_data);

    if (head == null) {
        head = new_node;
        return;
    }

    Node last = head;
    while (last.next != null) {
        last = last.next;
    }

    last.next = new_node;
}
```

## Python

```
def insert_at_end(self, new_data):
    new_node = Node(new_data)

    if not self.head:
        self.head = new_node
        return

    last = self.head
    while last.next:
        last = last.next

    last.next = new_node
```

# Deletion of Element in Linked List

- Traverse list to find node to be deleted
- Make previous' next pointer skip the target node
- Delete the target node and free the allocated memory

# Deletion of Element in Linked List

## Java

```
public void deleteNode(int key) {
    Node temp = head, prev = null;

    if (temp != null && temp.data == key) {
        head = temp.next;
        return;
    }

    while (temp != null && temp.data != key) {
        prev = temp;
        temp = temp.next;
    }

    if (temp == null) return;

    prev.next = temp.next;
}
```

## Python

```
def delete_node(self, key):
    temp = self.head

    prev = None
    if temp and temp.data == key:
        self.head = temp.next
        return

    while temp and temp.data != key:
        prev = temp
        temp = temp.next

    if temp is None:
        return
    prev.next = temp.next
```

# Singly Linked List – Java Implementation Part 1

```
class Node {
    public int data;
    public Node next;
    public Node(int data) {
        this.data = data; this.next = null;
    }
}

public class LinkedList {
    public Node head = null;
    public LinkedList() { head = null; }

    public void insertAtBeginning(int data) {
        Node new_node = new Node(data);
        new_node.next = head;
        head = new_node;
    }

    public void insertAtEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        Node last = head;
        while (last.next != null) { last = last.next; }
        last.next = newNode;
    }
}
```

```
public void insertAtPosition(int data, int position) {
    if (position < 1)
        return;
    Node newNode = new Node(data);
    if (position == 1) {
        newNode.next = head;
        head = newNode;
        return;
    }
    Node curr = head;
    for (int i = 1; i < position - 1 && curr != null; i++) {
        curr = curr.next;
    }
    if (curr == null)
        return;
    newNode.next = curr.next;
    curr.next = newNode;
}

public void deleteAtBeginning() {
    if (head == null)
        return;
    head = head.next;
}
```

# Singly Linked List – Java Implementation Part 2

```
public void deleteAtEnd() {
    Node secondLast = head;
    while (secondLast.next.next != null) {
        secondLast = secondLast.next;
    }
    secondLast.next = null;
}

public void deleteAtPosition(int position) {
    Node temp = head, prev = null;
    if (temp == null)
        return;
    if (position == 1) {
        head = temp.next;
        return;
    }
    for (int i = 1; temp != null && i < position; i++) {
        prev = temp;
        temp = temp.next;
    }
    if (temp != null) {
        prev.next = temp.next;
    }
    else {
        System.out.println("Data not present");
    }
}
```

```
public void traverse() {
    int pos = 1;
    Node curr = head;
    while (curr != null) {
        System.out.print("    -> " + curr.data + " @P"+pos);
        curr = curr.next;
        pos++;
    }
    System.out.println();
}

public int length() {
    int count = 0;
    Node curr = head;
    while (curr != null) {
        count++;
        curr = curr.next;
    }
    return count;
}
```

# Singly Linked List – Java Implementation Part 3

```
public int get(int position) {
    if (position < 1)
        return -1;
    if (position == 1) {
        return head.data;
    }
    Node curr = head;
    for (int i = 1; i < position && curr != null; i++) {
        curr = curr.next;
    }
    if (curr == null)
        return -1;
    return curr.data;
}

public boolean find(int data) {
    Node curr = head;
    while (curr != null) {
        if (curr.data == data)
            return true;
        curr = curr.next;
    }
    return false;
}
```

```
public static void main(String[] args) {
    LinkedList list = new LinkedList();

    System.out.println("Inserting 5 at Beginning");
    list.insertAtBeginning(5);
    list.traverse();

    System.out.println("Inserting 1 at Position 2");
    list.insertAtPosition(1, 2);
    list.traverse();

    System.out.println("Length = "+list.length());

    System.out.println("Deleting at End");
    list.deleteAtEnd();
    list.traverse();

    System.out.println("Getting Data");
    System.out.println(" Position:1");
    System.out.println("   Data:"+list.get(1));
}
```



# Singly Linked List – Python Implementation Part 1

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insertAtBeginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insertAtEnd(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node

    def insertAtPosition(self, data, index):
        if index == 1:
            self.insertAtBeginning(data)
            return
        position = 1
        current_node = self.head
        while current_node is not None and position + 1 != index:
            position += 1
            current_node = current_node.next
        if current_node is not None:
            new_node = Node(data)
            new_node.next = current_node.next
            current_node.next = new_node
        else:
            print("Index not present")

    def deleteAtBeginning(self):
        if self.head is None:
            return
        self.head = self.head.next
```

# Singly Linked List – Python Implementation Part 2

```
def deleteAtEnd(self):
    if self.head.next is None:
        self.head = None
        return
    current_node = self.head
    while current_node.next and current_node.next.next:
        current_node = current_node.next
    current_node.next = None
```

```
def deleteAtPosition(self, position):
    temp = self.head
    prev = None
    if position == 1:
        self.head = temp.next
        return
    for i in range(1, position):
        prev = temp
        temp = temp.next
    if temp is None:
        print("Data not present")
    if temp is not None:
        prev.next = temp.next
```

```
def traverse(self):
    temp = self.head
    while temp:
        print(temp.data, end=' ')
        temp = temp.next
    print()
```

```
def length(self):
    size = 0
    current_node = self.head
    while current_node:
        size += 1
        current_node = current_node.next
    return size
```

# Singly Linked List – Python Implementation Part 3

```
def get(self, index):
    if index == 1:
        return self.head.data
    curr = self.head
    position = 1
    while curr is not None and curr.next is not None and position != index:
        position += 1
        curr = curr.next
    if curr is None:
        return -1
    return curr.data

def find(self, data):
    if self.head.data == data:
        return True
    curr = self.head
    while curr is not None and curr.next is not None:
        if curr.data == data:
            return True
        curr = curr.next
    return False

if __name__ == "__main__":
    llist = LinkedList()
    llist.insertAtBeginning(5)
    llist.traverse()
    llist.insertAtBeginning(3)
    llist.traverse()
    llist.insertAtBeginning(4)
    llist.traverse()
    llist.insertAtPosition(1, 2)
    llist.traverse()
    print("length=", llist.length())
    llist.deleteAtEnd()
    llist.traverse()
    print("Postion=1,Data=", llist.get(1))
```

# Linked List Common Problems

- **Reverse a Linked List**

- Reverse the order of nodes in a linked list.

- **Detect a Cycle**

- Determine if a linked list contains a cycle where a node points back to a previous node in the list which created a circular cycle in the list
- Floyd's Cycle Finding Algorithm (aka "tortoise and hare" algorithm) uses slow and fast pointers to move through the list
- Slow pointer moves one node at time, while fast pointer moves two nodes at time.
- If there is a cycle, the fast pointer will eventually catch up to the slow pointer.

- **Merge Two Sorted Lists**

- Merge two sorted linked lists into a single sorted list.

# Reverse a Linked List

## Java

```
public Node reverseList(Node head) {  
    Node prev = null;  
    Node current = head;  
    Node next = null;  
    while (current != null) {  
        next = current.next;  
        current.next = prev;  
        prev = current;  
        current = next;  
    }  
    return prev;  
}
```

## Python

```
def reverse_list(self):  
    prev = None  
    current = self.head  
    while current:  
        next_node = current.next  
        current.next = prev  
        prev = current  
        current = next_node  
    self.head = prev
```

# Detect a Cycle in Linked List

## Java

```
public boolean detectCycle(Node head) {
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            return true;
        }
    }
    return false;
}
```

## Python

```
def detect_cycle(self):
    slow = fast = self.head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True
    return False
```

# Merge Two Sorted Linked Lists

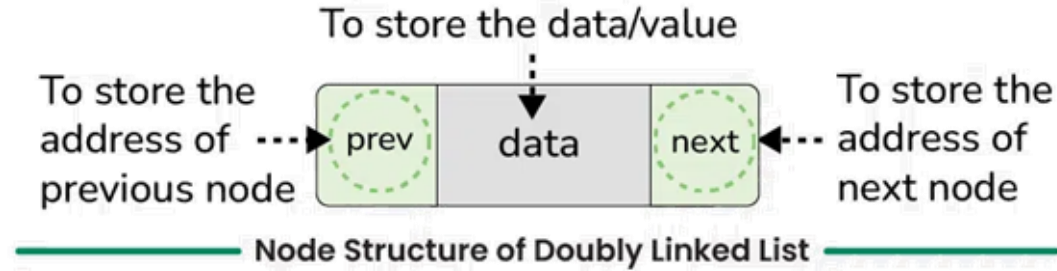
## Java

```
public Node mergeTwoSortedList(Node head1, Node head2) {
    Node temp = new Node(0);
    Node current = temp;
    while (head1 != null && head2 != null) {
        if (head1.data <= head2.data) {
            current.next = head1;
            head1 = head1.next;
        } else {
            current.next = head2;
            head2 = head2.next;
        }
        current = current.next;
    }
    if (head1 != null) {
        current.next = head1;
    } else {
        current.next = head2;
    }
    return temp.next;
}
```

## Python

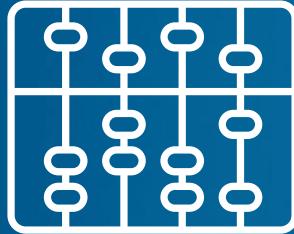
```
def merge_two_sorted_lists(head1, head2):
    temp = Node(0)
    current = temp
    while head1 and head2:
        if head1.data <= head2.data:
            current.next = head1
            head1 = head1.next
        else:
            current.next = head2
            head2 = head2.next
        current = current.next
    current.next = head1 if head1 else head2
    return temp.next
```

# Doubly Linked List Operations





# Complexity & Comparison



# Arrays vs Linked Lists

| Feature           | Array                   | Linked List                   |
|-------------------|-------------------------|-------------------------------|
| Memory allocation | Static                  | Dynamic                       |
| Memory usage      | No extra space required | Need extra space for pointers |
| Ease of use       | Simpler                 | More complex                  |

# Time Complexity Comparison

| Operation              | Array  | Linked List |
|------------------------|--------|-------------|
| Access                 | $O(1)$ | $O(n)$      |
| Search                 | $O(n)$ | $O(n)$      |
| Insertion at beginning | $O(n)$ | $O(1)$      |
| Insertion at end       | $O(1)$ | $O(n)$      |
| Deletion at beginning  | $O(n)$ | $O(1)$      |
| Deletion at end        | $O(1)$ | $O(n)$      |

# Advantages & Disadvantages

|               | Arrays                                                                                                                                               | Linked Lists                                                                                                                                   |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Advantages    | <ul style="list-style-type: none"><li>• Fast access time</li><li>• Simple and easy to use</li><li>• Good cache locality</li></ul>                    | <ul style="list-style-type: none"><li>• Dynamic size</li><li>• Efficient insertion and deletion</li><li>• Flexible memory allocation</li></ul> |
| Disadvantages | <ul style="list-style-type: none"><li>• Fixed size</li><li>• Inefficient insertions and deletions</li><li>• Wasted space for sparse arrays</li></ul> | <ul style="list-style-type: none"><li>• Sequential access</li><li>• Extra memory for storing links</li><li>• Not cache-friendly</li></ul>      |